



Smart Inventory Management System with AI-Based Business Insights

Module: Enterprise Architecture (HNDIT4232)

Student Name: L.M.A.Lakshani

Student ID: BAD-IT-2324-F-016

Table of Contents

1. Introduction.....	3
2. Problem Statement.....	4
3. Objectives.....	5
4. Literature Review.....	6
5. System Analysis.....	7
6. System Design.....	8
7. Database Design.....	10
8. AI Model Explanation.....	12
9. Implementation.....	14
10. Testing.....	16
11. Results & Discussion.....	17
12. Conclusion.....	18
13. References.....	19

1. Introduction

Inventory management plays a critical role in the success of any retail business.

Supermarkets and retail stores must maintain the right products in the right quantities to meet customer demand while minimizing operational costs. Poor inventory control can result in stock shortages, lost sales, excess inventory, increased storage costs, and wastage of perishable goods. In many small and medium-sized businesses, inventory decisions are still made manually based on experience, intuition, and historical observations, which can lead to inaccuracies and inefficient stock management.

Modern organizations such as Amazon and Walmart leverage Artificial Intelligence (AI) and data analytics to optimize inventory operations. By analyzing historical sales patterns and stock data, these systems can forecast demand, identify trends, predict shortages, and provide automated recommendations that support better business decisions.

This project, the **Smart Inventory Management System (SIMS)**, is a Java-based web application designed for supermarkets and retail stores. The system provides comprehensive functionality for managing products, inventory, and sales transactions while integrating AI and Machine Learning (ML) capabilities to enhance decision-making.

Intelligent features such as **restock prediction**, **Fast/Slow/Dead Stock analysis**, **sales trend forecasting**, and **automated inventory alerts** transform raw business data into actionable insights. For example, the system can recommend timely stock replenishment, identify underperforming products, and highlight sales patterns such as increased demand during weekends.

By combining traditional inventory management functions with AI-powered analytics, SIMS helps businesses improve inventory control, reduce losses, optimize stock levels, and make data-driven decisions that enhance overall operational efficiency.

2. Problem Statement

Many small and medium-sized businesses still rely on manual inventory management practices, which often lead to inefficiencies, inaccurate stock control, and poor decision-making. Without intelligent monitoring and forecasting, businesses face several common challenges:

- **Reactive Restocking** – Products are often reordered only after stock has been depleted, resulting in lost sales and dissatisfied customers.
- **Overstocking** – Excess inventory increases storage costs, ties up working capital, and may lead to unnecessary waste.
- **Dead Stock** – Slow-moving or non-selling products remain unnoticed, occupying valuable shelf and warehouse space.
- **Expiry Losses** – Perishable products may expire before action is taken, causing financial losses and inventory waste.
- **Lack of Forecasting** – Traditional systems provide limited insight into future demand, making it difficult to identify sales trends, seasonal patterns, or peak purchasing periods.

To address these challenges, the proposed **Smart Inventory Management System** combines inventory management with AI-powered analytics. By analyzing historical sales and stock data, the system can predict restocking needs, identify fast- and slow-moving products, forecast future sales trends, and generate proactive alerts. This enables businesses to make data-driven decisions, optimize inventory levels, reduce losses, and improve overall operational efficiency.

3. Objectives

The objectives of this project are to:

- Develop a complete **Java-based web application** for inventory and sales management.
- Implement **secure user authentication and authorization** with Admin and Staff roles.
- Manage products and inventory through functions such as adding, updating, deleting, and searching products.
- Record sales transactions, generate invoices, and maintain sales history.
- Generate business reports, including daily sales, monthly sales, and stock status reports.
- Implement **four AI/ML-powered features** with clear and explainable logic.
- Provide data visualization through dashboards and charts for business insights.
- Integrate the Java application with a **Python-based AI/ML REST API** for intelligent analysis and forecasting.

These objectives aim to create a smart inventory management solution that improves operational efficiency and supports data-driven decision-making.

4. Literature Review

Modern businesses use **data-driven inventory management** to improve efficiency, reduce costs, and maintain optimal stock levels. Major retailers such as Amazon and Walmart employ demand forecasting and inventory optimization techniques to minimize stock shortages and excess inventory.

This project incorporates several established AI and analytical techniques:

- **Linear Regression** – A supervised machine learning algorithm used to forecast future sales based on historical data. It is widely used because it is simple, efficient, and easy to interpret.
- **K-Means Clustering** – An unsupervised learning technique that groups similar products together. In this project, it is used to classify products as **Fast-Moving, Slow-Moving, or Dead Stock**, supporting inventory planning and stock control.
- **Rule-Based Expert Systems** – Business rules are used to automate decisions and generate alerts, such as notifying users when stock levels fall below reorder thresholds. These systems are reliable, transparent, and easy to maintain.

By combining **machine learning (scikit-learn)** with **rule-based AI**, the Smart Inventory Management System provides both predictive analytics and explainable decision support. This hybrid approach improves inventory control, forecasting accuracy, and overall business decision-making.

References related to these techniques are provided in **Section 13**.

5. System Analysis

5.1 Actors

- **Admin** – full access: manage products (including delete), inventory, sales, reports and AI insights.
- **Staff** – restricted access: record sales, view stock and reports (cannot delete products).

5.2 Functional Requirements

1. User authentication (login/logout) with roles.
2. Product management – add, update, delete, search.
3. Inventory management – add stock, adjust quantity, view stock, low-stock alerts.
4. Sales management – record sales, generate invoices, store history.
5. Reports – daily sales, monthly sales, product stock.
6. AI features – restock prediction, fast/slow/dead analysis, trend forecast, intelligent alerts.

5.3 Non-Functional Requirements

- **Usability** – clean dashboard, charts, responsive Bootstrap UI.
- **Reliability** – AI works even if the Python service is offline (rule-based fallback).
- **Security** – password hashing (BCrypt), role-based authorisation.
- **Maintainability** – layered architecture (controllers, services, repositories).
- **Performance** – indexed database queries and SQL-level aggregation.

5.4 Feasibility

The system uses free, open-source, widely-supported technologies (Java, Spring Boot, MySQL, Python) and runs on a normal PC, making it technically and economically feasible.

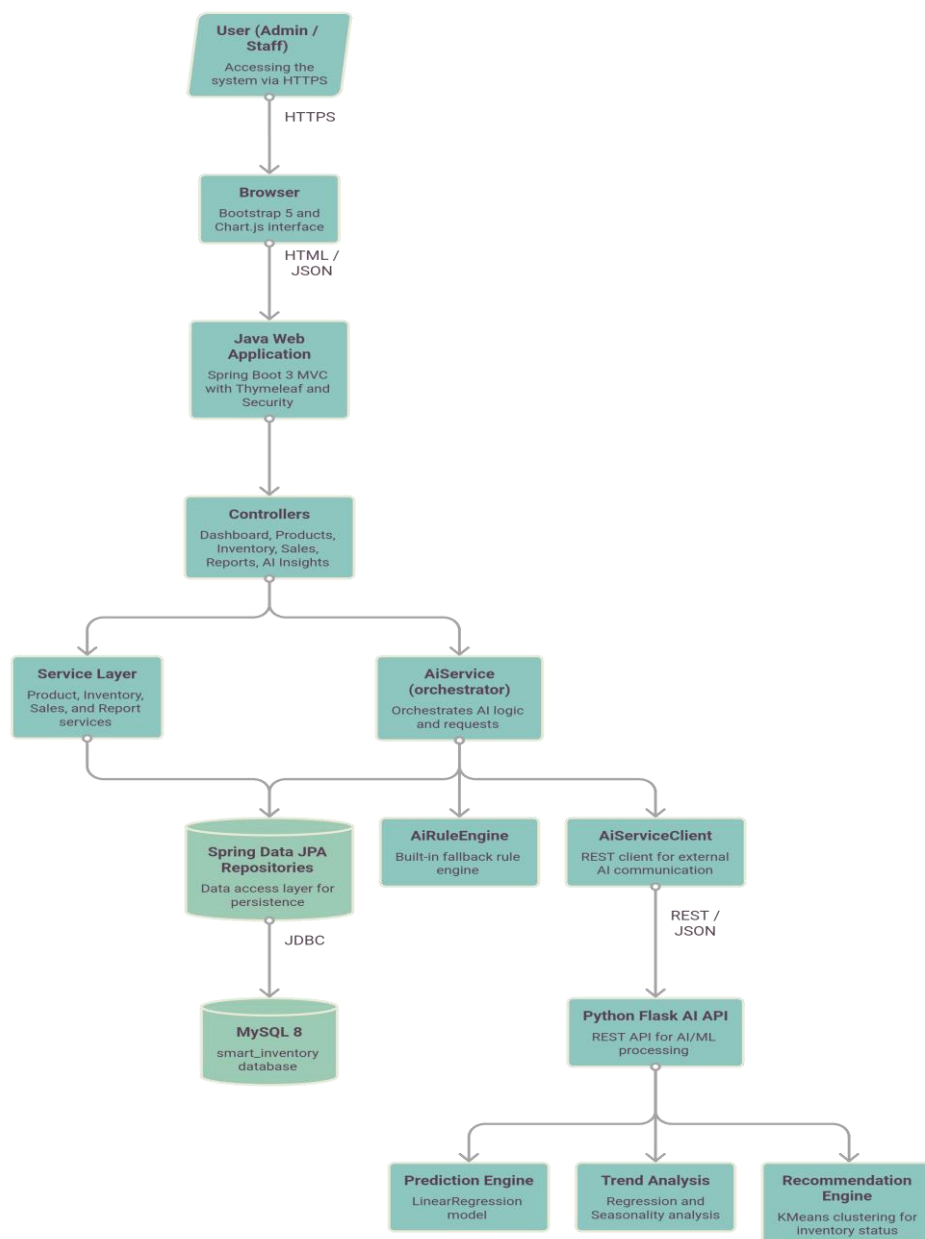
6. System Design

6.1 Architecture

The system follows a **layered architecture** plus a separate **AI microservice**:

If the Python service is unavailable, the Java app falls back to a built-in **rule-based AI engine**, so insights are always produced.

High-level Architecture of Smart Inventory System



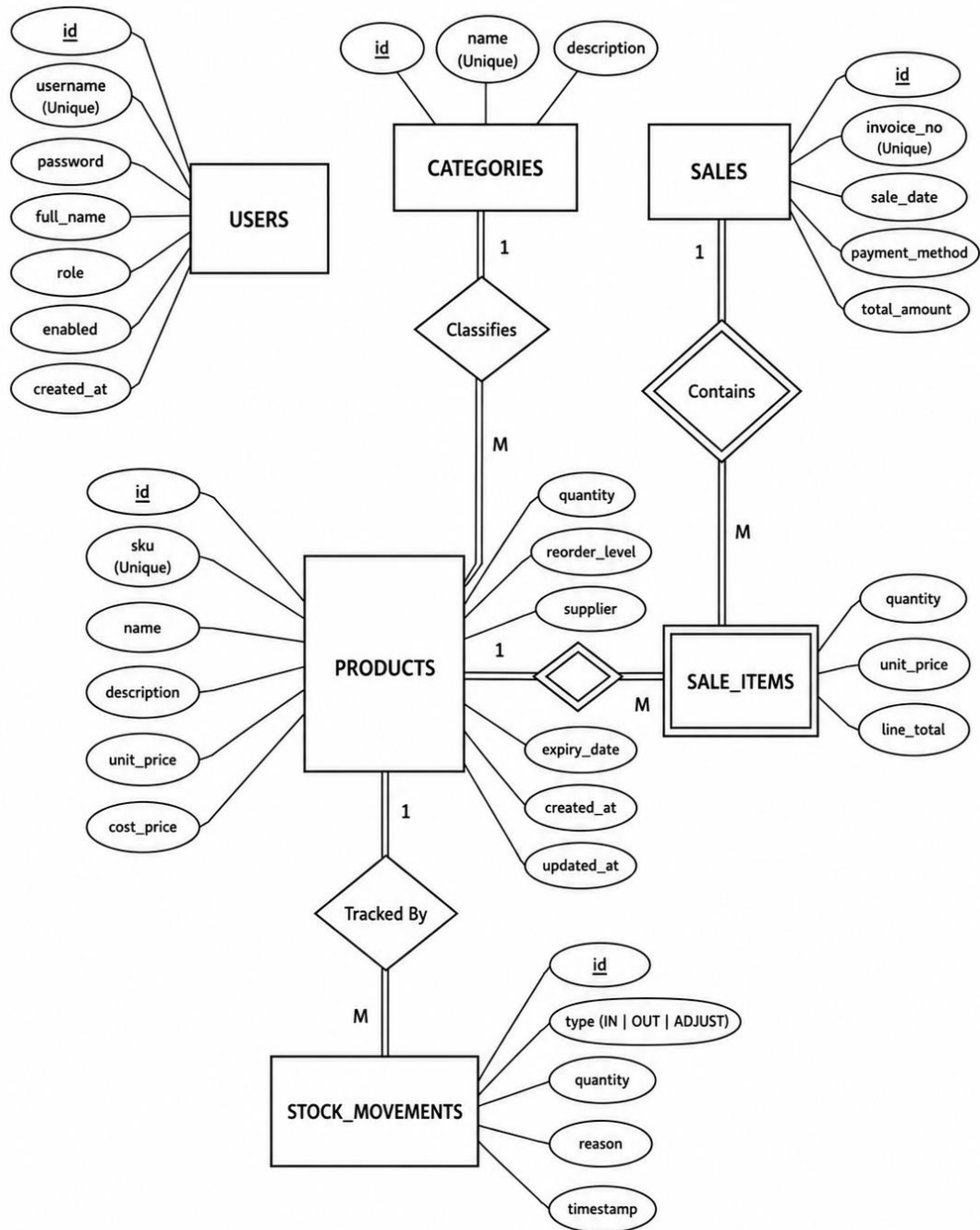
6.2 Technology Stack

Layer	Technology
Presentation	Thymeleaf, Bootstrap 5, Chart.js
Web / Backend	Java 17, Spring Boot 3.3, Spring MVC, Spring Security
Persistence	Spring Data JPA, Hibernate, MySQL 8
AI / ML	Python 3, Flask, scikit-learn, NumPy
Build	Maven

6.3 Module Design

- **Controllers** – one per page (Dashboard, Product, Inventory, Sales, Report, AI Insights) plus a JSON ChartDataController for charts.
- **Services** – ProductService, InventoryService, SalesService, ReportService, and AiService (orchestrates AI).
- **AI layer** – AiRuleEngine (Java, rule-based), AiServiceClient (calls Flask), and Python engines/ (ML models).
- **Repositories** – Spring Data JPA interfaces for each entity.

7. Database Design



7.1 Relationships

- One **Category** has many **Products** (1:N).
- One **Sale** has many **Sale Items** (1:N).
- One **Product** appears in many **Sale Items** (1:N).
- One **Product** has many **Stock Movements** (1:N).

The database smart_inventory contains six tables:

Table	Purpose
users	Login accounts and roles (Admin/Staff)
categories	Product categories (Dairy, Bakery, ...)
products	Product details, price, stock, reorder level, expiry
sales	Sales transactions (invoices)
sale_items	Line items within each sale
stock_movements	Audit log of every stock change (IN/OUT/ADJUST)

7.2 Data Volume

To satisfy the minimum data requirement, the system auto-seeds **64 products** and **~1,400 sales records** spanning 150 days of history, including realistic patterns: weekend spikes, fast/slow/dead movers, low-stock and expiring items.

8. AI Model Explanation

The system implements **four AI features**. Three use **machine learning** (scikit-learn) with an equivalent **rule-based** fallback in Java; the alerts are rule-based by design.

8.1 Feature 1 — Smart Restock Prediction

Goal: predict when each product will run out and how much to reorder. **Logic:**

1. Build each product's **daily sales series** over a 90-day window.
2. Estimate **future daily demand d**:
 - a. *ML mode*: LinearRegression fits the sales trend; the result is blended with the recent 14-day average.
 - b. *Rule mode*: $d = \text{total units sold} \div \text{number of days}$.
3. **Days to stockout** = $\text{current stock} \div d$.
4. **Recommended restock quantity** = $(d \times 30) - \text{current stock}$ (aim for 30 days of cover).
5. **Urgency**: HIGH (≤ 3 days or below reorder level), MEDIUM (≤ 7), LOW (≤ 14), else OK.
 - *Example: "Fresh Milk 1L may run out within 4 days. Recommended restock quantity: 60 units."*

8.2 Feature 2 — Fast / Slow / Dead Moving Analysis

Goal: classify products by sales performance. **Logic:**

- A product is **DEAD** if it has never sold or has not sold in the last 30 days.
- *ML mode*: the remaining products are grouped with **K-Means clustering** on (*units sold, recency*); the higher-selling cluster is **FAST**, the other **SLOW**.
- *Rule mode*: the busiest 25% are **FAST**, the rest **SLOW**.
- Displayed as a **doughnut chart** plus fast-mover and dead-stock tables.

8.3 Feature 3 — Sales Trend Analysis & Forecast

Goal: understand and predict revenue trends. **Logic:**

1. Build the **daily revenue series** (missing days filled with 0).
2. LinearRegression finds the trend **slope** → growth rate %.
3. A **weekday seasonal profile** (average revenue per weekday) is computed.
4. Each future day is forecast as $0.5 \times \text{trend} + 0.5 \times \text{weekday average}$.
5. Human-readable insights are generated automatically.

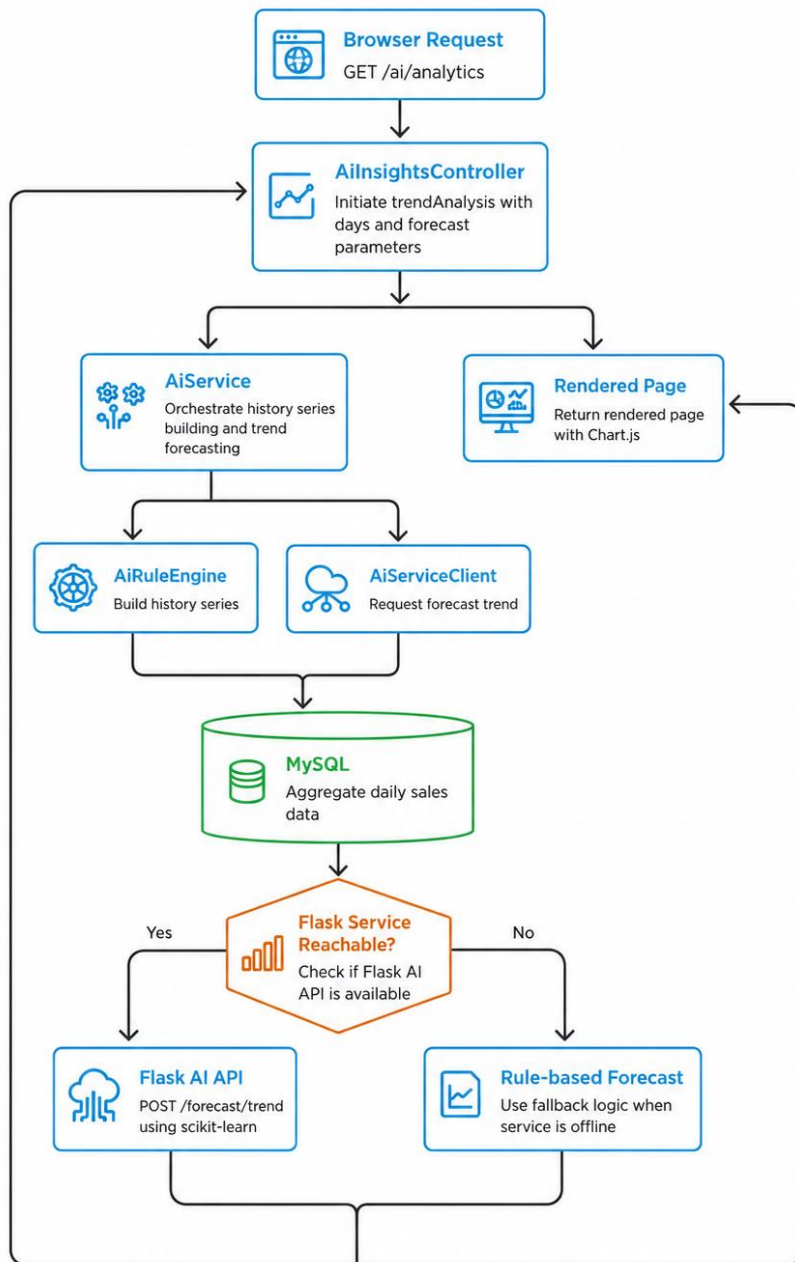
Example: "Sales increase on weekends (about 45% higher than weekdays)."

8.4 Feature 4 — Intelligent Alerts

Transparent business rules over live data:

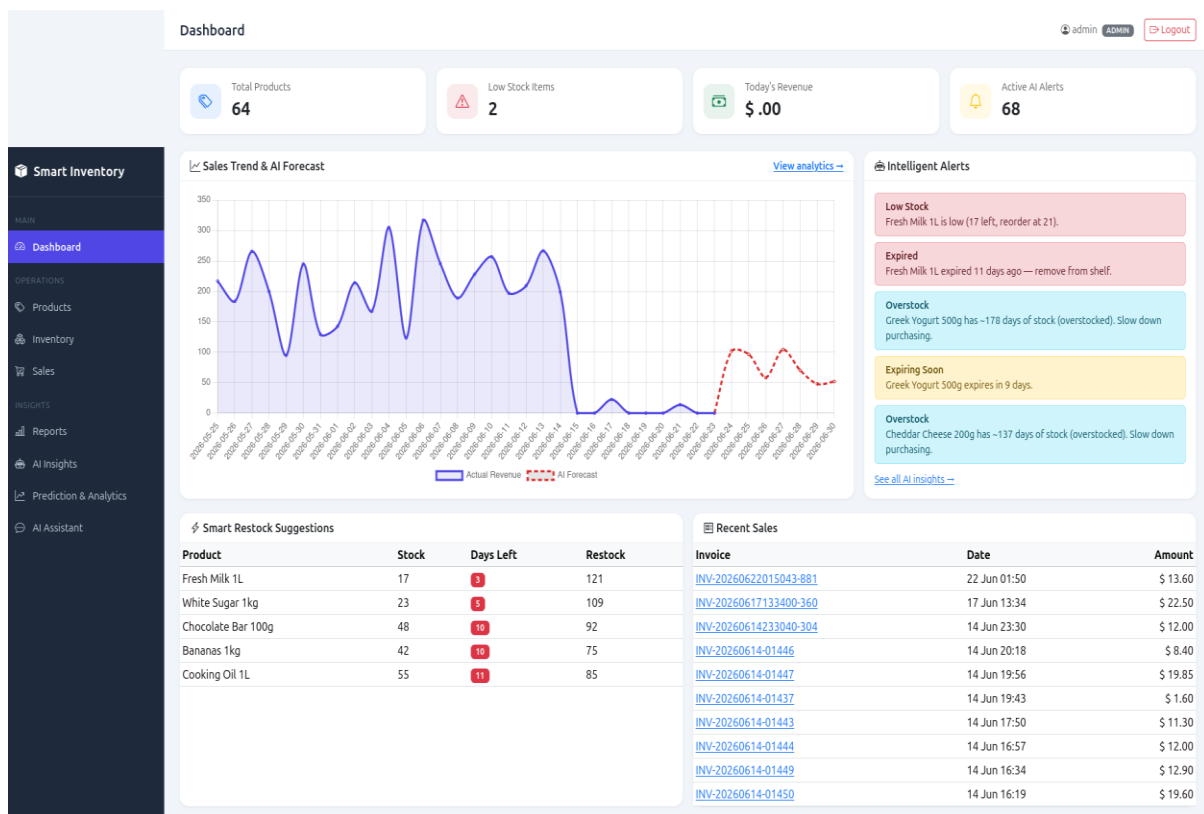
- **Low stock:** quantity $\leq$ reorder level.
- **Overstock:** stock cover greater than 60 days at current demand.
- **Sudden sales drop:** this week's revenue $\geq$ 30% lower than last week's.
- **Expiry:** perishable product expiring within 14 days (or already expired).

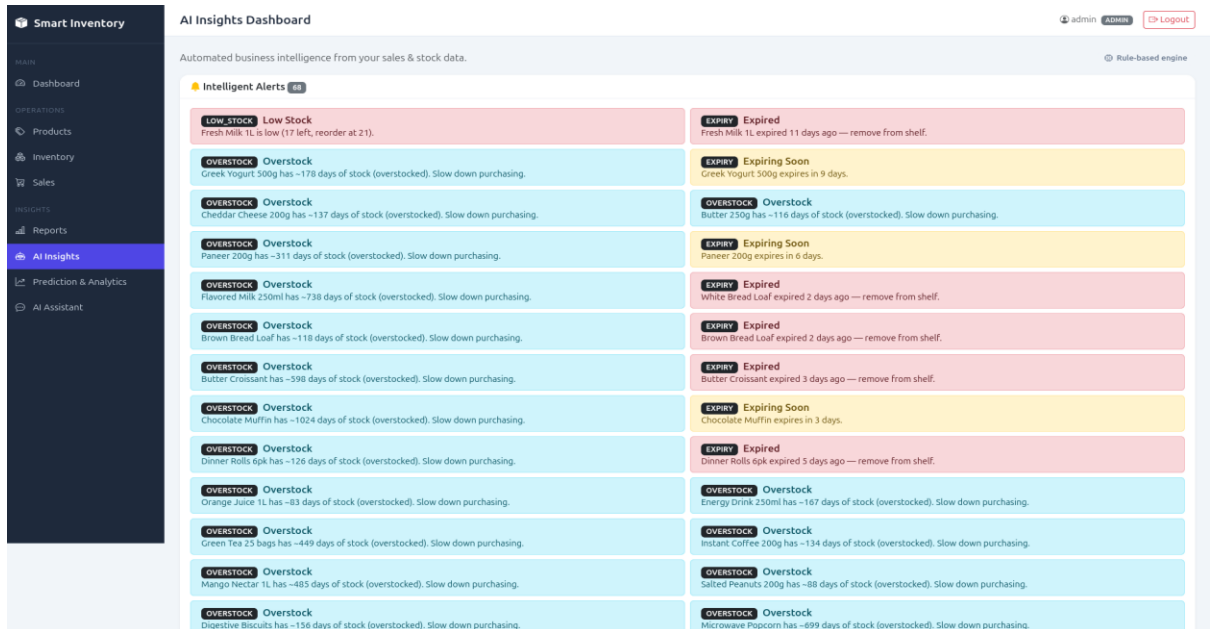
AI Analytics Request Flow



9. Implementation

- **Backend:** Java 17 + Spring Boot 3.3 (Spring MVC, Spring Security, Spring Data JPA). Passwords are hashed with **BCrypt**; access is role-based.
- **Frontend:** Thymeleaf templates styled with Bootstrap 5; charts drawn with **Chart.js** (Sales Trend, Product Performance, Revenue Analysis, AI Prediction).
- **AI service:** a Python **Flask** API exposing /predict/restock, /forecast/trend and /analyze/movement, implemented with **scikit-learn** (LinearRegression, KMeans).
- **Integration:** AiServiceClient (Spring RestTemplate) calls the Flask API over REST; on any failure it returns empty and the Java AiRuleEngine is used instead.
- **Data seeding:** a DataSeeder populates users, the catalogue and the sales history on first run.





10. Testing

#	Test Case	Steps	Expected Result	Status
1	Login (valid)	Login as admin/admin123	Dashboard shown	Pass
2	Login (invalid)	Wrong password	"Invalid username or password"	Pass
3	Role check	Login as staff	Delete button hidden / blocked	Pass
4	Add product	Fill product form, save	Product appears in list	Pass
5	Search product	Type a name/SKU	Matching products shown	Pass
6	Add stock	Add quantity in Inventory	Stock increases, movement logged	Pass
7	Low-stock alert	Reduce stock below reorder	Product flagged LOW + alert	Pass
8	Record sale	Add items, complete sale	Invoice generated, stock reduced	Pass
9	Reports	Open Reports	Daily/monthly/stock + charts shown	Pass
10	Restock prediction	Open AI Analytics	Days-to-stockout + restock qty shown	Pass
11	Trend forecast	Open AI Analytics	Forecast line + insights shown	Pass
12	Movement analysis	Open AI Insights	Fast/Slow/Dead chart populated	Pass
13	AI fallback	Stop Flask service	App still shows AI (rule-based)	Pass

11. Results & Discussion

The completed system meets all functional requirements:

- It correctly **records sales**, **updates stock**, and **generates invoices**.
- The **dashboard** and **reports** display accurate daily/monthly figures with four interactive charts.
- The AI features produce sensible, useful output on the seeded data:
 - Fast movers such as Fresh Milk, Cola and Bananas are identified.
 - Dead stock such as Glass Cleaner and Body Lotion is flagged.
 - Low-stock products generate restock predictions with recommended quantities.
 - The trend analysis detects the weekend sales spike that was built into the data.

The **dual ML + rule-based design** proved valuable: insights remain available whether or not the Python service is running, improving reliability. A limitation is that Linear Regression assumes broadly linear demand; products with strong seasonality would benefit from more advanced models.

12. Conclusion

The **Smart Inventory Management System** successfully demonstrates a modern Java-based web application that combines traditional inventory management with AI-driven analytics. The system efficiently manages products, stock levels, categories, sales transactions, and inventory movements while providing valuable business insights for better decision-making.

The integration of **Machine Learning (scikit-learn)** and rule-based intelligence enables key features such as **restock prediction**, **Fast/Slow/Dead Stock analysis**, **sales trend forecasting**, and **intelligent inventory alerts**. These capabilities help businesses optimize stock management, reduce inventory costs, and improve operational efficiency. Interactive dashboards and charts further enhance data visualization and analysis.

Overall, the project demonstrates the practical application of AI and ML in inventory management and provides a scalable foundation for future enhancements. Future improvements may include advanced forecasting models such as **ARIMA** or **Prophet**, barcode and QR code scanning, product recommendation systems, mobile application support, supplier integration, and an AI-powered chatbot assistant for inventory-related decision support.

13. References

1. Apache. *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>
2. scikit-learn developers. *scikit-learn: Machine Learning in Python*. <https://scikit-learn.org/>
3. Pallets. *Flask Documentation*. <https://flask.palletsprojects.com/>
4. Oracle. *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/>
5. Chart.js. *Chart.js Documentation*. <https://www.chartjs.org/docs/>
6. *[Add 1–2 textbook or journal references your course recommends, e.g. on inventory management / demand forecasting.]*

Appendix — Demo Accounts

Role	Username	Password
Admin	admin	admin123
Staff	staff	staff123